滇池学院校园 AI 客服

项目报告

项目名称	校园 AI 客服（滇池学院）
技术路线	RAG（检索增强生成）+ 双路混合检索 + 覆盖率闸门
交付形态	Web 对话页、官网悬浮挂件、HTTP 接口、管理后台
知识库规模	9 份资料、98 个知识片段
代码规模	23 个 Python 文件、约 2150 行
承担人	许心安
学号	
学院 / 专业	
指导教师	
报告日期	2026 年 10 月 11 日

一、项目背景与目标

1.1 背景

高校招生季的咨询有几个很明显的特点：问题高度重复，学费多少、宿舍几人间、怎么报到、奖学金怎么申请、多少分能上，翻来看去就是这些；时间高度集中，出分后的两周内咨询量会集中涌入；回答口径必须一致，同一个问题不能两个人给出两种说法；而招生办的电话线是有限的。

把这些问题交给一个什么都会聊的大模型是危险的。模型会用常识补全，而招生、学费、学籍这类信息一旦编造，考生和家长会当真。反过来，做一个只能查资料、查不到就老实说查不到的系统，虽然能力看起来弱，但在校园场景里恰恰是正确的取舍。

1.2 项目目标

据此确定四条可衡量的目标。

目标	具体要求
只依据学校资料作答	不允许用模型常识补全，不编造电话、网址、金额、分数线
每条回答可溯源	答案标注引用编号，可展开查看原文出自哪份资料的哪一节
答不上来要老实说	宁可回答没查到并引导到官方渠道，也不硬答，用跑题误答率度量
校方能自己维护	不改代码、不重启服务，上传文件或网页后台即可更新知识库

二、技术方案

2.1 为什么选 RAG 而不是微调或直接提问

招生信息每年都在变，微调的成本和滞后性都不可接受。RAG 把事实和表达分开：事实放在可核查的知识库里，表达交给模型。这样做最坏情况下把模型拿掉也仍然能用，因为答案的原料始终来自学校自己的资料。

方案	资料更新	可溯源	编造风险	成本
直接问大模型	无意义	无	高	按量付费
微调模型	需重新训练	无	中	高
RAG（本项目）	改文件即生效	带出处	低	可降到零

2.2 系统架构

一次提问在系统内部经过六道处理，任何一步的判断依据都是学校自己的资料。

环节	处理	说明
安全护栏	越权、跑题、提示词注入检测	命中则礼貌拒绝，不进入检索
查询加权	中文分词 + 未登录词子词回退	如“学校地址”拆为“学校”加“地址”
双路召回	BM25 关键词检索与 LSA 语义向量检索并行	每路各召回 12 条候选
RRF 融合	两路排名做倒数加权合并	参数 rrf_k 取 60
双闸门过滤	融合分数与 IDF 加权覆盖率双重判定	任一不达标即走兜底话术
拼装与生成	学校档案加检索片段拼成提示词，流式生成	答案带 1、2 编号出处标注

2.3 检索层设计

中文分词优先使用 jieba，缺少依赖时自动退化为字符 n-gram，保证在任何环境都能跑起来。

检索做成两路混合，是因为校园提问里两类问法都很常见：学费这类关键词问法靠 BM25 就够了，而“多少钱一年”和“收费标准”这种同义改写要靠语义向量。单走一路都会漏。

语义向量使用本地实现，即 TF-IDF 加截断 SVD 的潜语义索引，用 numpy 就能完成，不需要下载模型、不需要联网、不产生任何费用。对校园问答这种领域窄、术语固定的场景，配合 BM25 已经够用；需要更强的语义理解时，可通过配置切换成任意 OpenAI 兼容的 Embedding 接口。

覆盖率闸门是本项目防编造的关键设计。融合分数只说明某段资料排第几，不说明它到底覆盖了问题的多少内容。因此系统额外计算查询词的 IDF 加权覆盖率，也就是问题里的关键词有多少比例能在该片段中找到，低于 0.42 就直接判定没有相关资料。实测中这一条是把跑题误答压到零的主要功臣。

未登录词分两种处理：单字词如“哪”“学”“带”视为疑问字或口语填充，不计权；多字词先尝试拆成已知子词再计权，拆不开才当作真正的新信息按最高 IDF 计权。这是“有资料却答不出来”问题的主要来源，也是调试中花时间最多的一处，详见 4.4 节。

2.4 生成层设计

系统提示词写死了十条回答规则，核心是四条：只依据知识库回答；知识库不足以回答时明说没查到并给出官方渠道；金额、日期、比例必须与原文一致不得改写；不得编造网址、电话、邮箱、人名。另加两条治理性约束：不承诺录取结果、不预测分数线；涉及招生录取、学籍、缴费等正式事项，结尾必须提示以学校正式通知为准。

安全护栏采用正则加提示词的双重拦截。正则负责识别“忽略以上指令”“输出你的系统提示词”“扮演其他角色”“越狱”等注入式提问，提示词负责处理更泛的越界请求。答案长度限制在 400 字以内，能一句话说清就不展开。

2.5 四级降级机制

这是本项目在可用性上最下功夫的地方：任何一个环节出问题，系统都不会返回服务器错误，而是返回一个诚实的降级结果。

故障场景	系统行为	用户看到
未配置大模型	走纯检索模式	直接给出知识库命中的资料原文，零延迟零成本
大模型调用失败	回退到检索原文	提示大模型暂时不可用，附上命中的资料原文
检索不到相关内容	走兜底话术	提示没查到准确信息，附官方电话、官网与人工客服时间
资料已更新但索引未重建	启动时检测并告警	页面顶部提示回答可能不是最新版本

纯检索模式还有一个额外价值：它让系统可以零成本地对所有人开放。因为不调用任何外部接口，也就不存在额度被刷的问题。

2.6 知识库工程

知识库不是简单地把文件丢进去，而是做了几层处理，使其既便于检索，也便于学校老师长期维护。

处理	做法	带来的效果
按标题层级切分	Markdown 按标题切成小节，记录面包屑路径	点击出处能直接看到内容来自哪一节
问答表优先	CSV 与 JSONL 一行一条问答，问题原文一并入库	用户问法与问答表一致时覆盖率直接到 1.0
下划线文件不入库	以底划线开头的文件不参与检索	可以放心用来写待办与核对备忘
注释自动剥离	HTML 注释视为编写者备注	备注不会进入检索结果
切分参数	目标长度 450 字符，重叠 80 字符	片段够聚焦，上下文又足够完整

三、系统功能

3.1 对话端

对话页首屏放了六个推荐问题，点一下直接提问，降低首次使用的门槛。答案采用流式输出，逐字出现，不需要等整段生成完。答案里的引用编号可以点击，展开显示该片段的原文，实现可核查；每条回答下方有赞和踩两个反馈按钮，写入日志供后续改进。页面顶部会根据运行状态显示提示，例如当前处于资料原文检索模式或索引未重建。底部固定免责声明，明确仅供参考、正式事项以学校文件为准。



图 1 对话界面：回答带引用编号，底部可展开查看原文

移动端按 390×780 做了适配，自动化检测横向溢出为 0 像素。



图 2 移动端布局

3.2 官网悬浮挂件

学校官网只需在任意页面结束标签前加一行脚本，右下角就会出现悬浮按钮，点开即对话。挂件内部以同源的 iframe 加载聊天页，避免与官网自身样式冲突；主题色、标题、左右位置都可通过属性配置。同时也支持用 iframe 直接嵌入整页。

```
<script src="https://你的域名/widget.js"
        data-api="https://你的域名"
        data-color="#0B4F9E"
        data-title="滇池学院智能客服"
        data-position="right" defer></script>
```

3.3 接口清单

对话类接口对外开放，供网页端、挂件以及第三方系统对接。

方法	路径	说明
GET	/api/meta	学校信息、主题色、推荐问题、索引状态

方法	路径	说明
POST	/api/chat	一次返回完整回答，适合第三方系统对接
POST	/api/chat/stream	流式返回，网页端使用
POST	/api/feedback	赞与踩的反馈
POST	/api/session/reset	清空会话上下文
GET	/api/health	健康检查

管理类接口需要请求头中携带管理令牌，覆盖运行统计、未命中问题清单、知识文件清单、重建索引、上传与删除知识文件、查看系统提示词、测试大模型连通性等八项能力。其中未命中问题清单是运营闭环的入口。

3.4 管理端与运营闭环

项目的运营思路是用没答上的问题驱动知识库迭代，形成一条不需要猜用户会问什么的改进路径。

```
用户在问什么   ->  查看热点问题统计
              ->  查看未命中问题清单
              ->  在对应资料里补一段，或往常见问答表加一行
              ->  重建索引（上传文件则自动重建）
              ->  跑回归测试确认没有退化
```

未命中清单这个接口看起来不起眼，但它是让知识库随时间自然变强的关键：维护者不需要猜用户会问什么，系统会把答不上来的问题直接列出来。

四、测试与实测数据

4.1 知识库与索引规模

项目	数值
知识文件	9 份（7 个 Markdown 专题、1 个常见问答 CSV、1 个不参与检索的核对清单）
知识片段	98 个
索引维度	97（本地 LSA，配置上限 256，实际受语料规模限制）
向量化方式	本地 TF-IDF 加截断 SVD，纯 numpy，不依赖外部服务

4.2 链路自检

项目自带自检脚本，覆盖分词、知识库读取、索引构建、检索回归四段，不需要联网即可运行。当前结果为六组检索用例全部通过。

```
1) 分词            jieba 可用
2) 读取知识库      片段数 98，文档数 9
3) 构建索引        provider=local  dim=97  chunks=98
4) 检索回归测试    6 / 6 全部通过
```

4.3 真实问题回归

新增的回归脚本用 22 个真实考生和家长提问作为正例，再加 5 个不该回答的问题作为负例，同时考核两项指标：召回衡量该答上的有没有答上，跑题误答衡量不该答的有没有硬答。

最终结果：22 个正例全部命中且命中正确片段，5 个负例全部走兜底。负例包括要求写英语作文、询问天气、询问爬虫写法、索取电影推荐，以及询问并不存在的火星校区招生计划。

固定知识库、只改覆盖率阈值做扫描，结果如下。

覆盖率阈值	召回	漏答	跑题误答
0.60	19 / 22	3	0
0.50	22 / 22	0	0

覆盖率阈值	召回	漏答	跑题误答
0.42（默认）	22 / 22	0	0
0.38	22 / 22	0	0
0.35	22 / 22	0	0
0.32	22 / 22	0	0
0.30	22 / 22	0	2
0.25	22 / 22	0	4

这张表说明两点：默认值 0.42 两侧都有安全余量，上到 0.50、下到 0.32 都保持全绿；而调低阈值换不来召回，却会立刻让跑题问题被硬答，所以它不该用来兜语言差异。

4.4 一次缺陷定位的完整过程

这一节记录一个真实问题的排查过程，因为它比结论更能说明工程方法。

现象是回归测试中“住宿费多少钱”被判定为漏答：系统返回的是学费那条问答，而真正该命中的住宿费条目被过滤掉了。

排查时打印了查询词权重，发现问句被切成“住宿费”和“钱”两个词。“钱”这个词在整个语料里只出现过一次，就在学费问答的“多少钱”里，因此拿到了全语料的最高 IDF 值 4.17；而“住宿费”虽然才是主题词，IDF 只有 2.22。

根因是小语料下 IDF 会失真：一个口语化的填充词因为罕见，权重压过了真正的主题词。后果是两个片段的判定被反了过来，错误的学费条目覆盖率 0.65 通过了闸门，正确的住宿费条目覆盖率 0.35 被拦掉。

先尝试了调低覆盖率阈值，从 0.42 依次降到 0.35 和 0.30。结果是召回几乎不涨，跑题误答从零涨到 2 至 4 个。又尝试给未登录词降权作为第二个旋钮，同样失败，召回不动而误答增加。这两个思路都被放弃，相应改动也已撤回，没有留在代码里。

最终改在知识库层，补一条同义问法的问答。因为问答表的问题原文会一并入库，新片段同时包含“住宿费”和“钱”两个查询词，覆盖率直接到 1.0，稳稳排在首位。

同样的方法还修掉了“滇池学院以前叫什么名字”和“学校什么时候改名的”这两个假阴性，资料里写的是前身是云南大学滇池学院、2023 年转设更名，与用户的口语问法对不上。

这一节的结论是：有资料却答不出来，绝大多数不是检索参数的问题，而是提问用词和资料用词不一致。补一条同义问答的收益远大于调阈值，而且不会牺牲防误答能力。

4.5 端到端性能

运行模式	实测耗时	说明
纯检索模式	0 毫秒	不调用外部接口，无网络往返，可承受高并发
大模型模式	约 3.4 秒	流式输出，用户看到首字的时间远小于此

4.6 前端验收

使用无头浏览器做自动化验收，检查控制台报错、页面异常、请求失败、移动端横向溢出等指标，并自动生成截图。当前结果为：控制台报错 0、页面异常 0、请求失败 0、移动端横向溢出 0 像素，推荐问题 6 个，出处标注正常渲染并可展开。



图 3 首次进入页面

五、工程化与部署

5.1 代码结构

后端按职责分层，每个模块只做一件事，便于单独替换与测试。

```
app/
  main.py           服务入口，启动时自动建索引，统一异常兜底
  settings.py       三层配置加载：配置文件、环境变量文件、环境变量
  textutil.py       中文分词、文本清洗、复合词拆分
  chunking.py       文档切片，支持 Markdown 标题感知、问答表、JSONL
  bm25.py           BM25 关键词检索实现
  embeddings.py     本地 LSA 向量与接口向量两种实现
  index_store.py    索引构建加载、混合检索、覆盖率闸门
  rag.py            问答主流程、安全护栏、降级策略
  prompt.py         系统提示词与兜底话术
  llm.py            大模型客户端，支持流式与非流式
  sessions.py       会话上下文，内存存储带过期清理
  ratelimit.py      按来源的滑动窗口限流
  store.py          问答日志、反馈与统计
  routers/          对话接口与管理接口
web/                聊天页与官网挂件
scripts/            建索引、自检、回归测试、命令行问答、界面验收
data/knowledge/     知识库，放学校资料的地方
data/index/         索引产物，可删除后重建
data/logs/          问答日志
```

5.2 启动方式

项目内置了全部运行依赖，不联网也能直接启动，脚本会自动建索引并启动服务。启动后打开本机地址即可使用。

```
run.ps1      Windows 一键启动
run.sh       Linux 与 macOS 一键启动
```

5.3 关键配置项

配置项	位置	当前值	作用
LLM_ENABLED	环境变量文件	0	大模型总开关，0 为纯检索模式，零成本对外
ADMIN_TOKEN	环境变量文件	已改为随机串	管理接口鉴权令牌
min_coverage	配置文件	0.42	覆盖率闸门，防硬答的关键参数
min_score	配置文件	0.18	融合分数下限

配置项	位置	当前值	作用
top_k / candidate_k	配置文件	5 / 12	送入模型的资料条数、每路召回条数
chunk_size 与重叠	配置文件	450 / 80	切片长度与重叠字符数
cors_origins	配置文件	已收紧	允许调用接口的站点白名单
rate_limit_per_minute	配置文件	30	每个来源每分钟的请求上限
history_turns	配置文件	4	携带进上下文的历史轮数

5.4 部署方案

容器化部署使用 Docker Compose 一条命令完成，知识库与索引目录以卷的形式挂载出来，改完资料重启容器或调用一次重建索引接口即可生效，无需重新构建镜像。

裸机部署在反向代理层有一个必须注意的点：Nginx 配置中要关闭响应缓冲，否则流式输出会退化成整段一次性出现。

局域网直接访问时，项目默认监听所有网卡，同一局域网的设备在放行防火墙端口后即可通过本机地址访问，无需额外部署。

5.5 版本控制与打包安全

项目已纳入 Git 管理，索引产物、日志、内置依赖和密钥文件均排除在版本库之外，并统一了换行符，避免 Windows 编辑器把 Linux 启动脚本改成 CRLF。

打包环节发现并修复了一个实质性风险：容器构建原本会把本地依赖目录和含真实密钥的配置文件一起复制进镜像层，而镜像层中的密钥事后无法清除。现已通过忽略文件全部排除。

另将容器默认进程数从 2 改为 1。会话上下文、限流计数、索引都保存在进程内存中，多进程会让同一会话的连续请求落到不同进程，导致多轮对话随机丢失上下文。需要横向扩展时，应先把会话与限流迁移到 Redis。

六、安全与合规

已实施的控制包括：管理接口全部要求令牌鉴权，缺失或错误返回未授权；跨域来源从通配收紧为学校官网域名白名单；密钥不进版本库也不进镜像；按来源限流，防止单点刷量；提示词层面禁止编造联系方式、禁止承诺录取结果，招生类问题强制提示以正式文件为准；知识库投喂约定中明确不得放入含个人隐私的文件。

仍存在两项已知风险，需要在正式开放前处理：对话接口本身没有登录，任何人只要知道地址即可提问，纯检索模式下没有费用风险，一旦切回大模型模式，开放局域网需配合限流或改用防火墙白名单；限流的来源识别直接采用了请求头中的转发地址，可被伪造绕过，若部署在反向代理之后，应当只信任来自代理的该头部。

七、项目亮点

亮点	说明
零依赖可离线运行	本地向量实现加内置依赖，无需下载模型、无需联网、无额外费用
四级降级机制	任一环节故障都不返回服务器错误，而是给出诚实的降级结果
答案可溯源	每条回答带引用编号，点开即可看到知识库原文，便于校方核查
双闸门防编造	融合分数与覆盖率双重判定，22 个真实问题下召回满分且跑题误答为零
零成本对外模式	纯检索模式响应 0 毫秒、无额度消耗，适合招生季向全体考生开放
运营闭环	未命中问题清单直接驱动知识库迭代，维护者不必猜用户会问什么
完整工程配套	一键启动、自检脚本、回归集、界面自动化验收、容器与反向代理方案、上线清单

八、已知限制与改进方向

限制	影响	改进方向
本地向量对同义改写弱于神经向量	口语化改写可能漏召回	切换为更强的向量接口，配置即可，需重建索引
会话与限流存于进程内存	多进程部署会丢上下文	迁移到 Redis
建索引在事件循环中执行	语料增大后重建期间会阻塞在线请求	移入线程池执行
追问的指代解析仅用上一轮问题补全	复杂复合指代仍可能落空	引入基于语义的追问改写
限流来源可被请求头伪造	开放部署时可能被绕过	仅在直连对端为可信代理时采信该头部
护栏依赖规则匹配	蹭到语料的代写类请求可能漏过	增加意图分类的二次判定
除自检与回归外暂无自动化测试	鉴权、上传、删除等路径缺少回归保护	补充针对管理接口的测试

九、结论

本项目完成了一个可直接部署的高校智能问答系统，覆盖从知识库工程、混合检索、防编造闸门、大模型生成到前端交互与运维管理的完整链路。

技术上的关键判断有三个：用 RAG 而不是微调，让事实与表达分离，资料更新即时生效；用覆盖率闸门而不是只靠相关度排序，把宁可说不知道落成可度量的机制；用双路混合检索而不是单一策略，同时接住关键词问法和语义问法。

工程质量上的关键判断也有三个：四级降级保证任何环节故障都不影响用户拿到结果；答案可溯源让校方敢于对外发布；未命中清单驱动的运营闭环让知识库能随使用自然变强。

实测数据支持了这些设计。22 个真实考生问题召回 22 例、跑题问题误答 0 例，链路自检 6 例全部通过，前端自动化验收无报错且移动端无溢出，纯检索模式响应

0 毫秒且零成本。项目当前状态为可试运行，上线前还需完成两项由校方主导的工作：核对知识库中的金额与电话等关键信息，以及由宣传部门审阅对外话术。

附录 A 复现步骤

```
# 1. 启动（自动建索引）
.\run.ps1

# 2. 不联网自检
$env:PYTHONPATH=".libs"; python scripts\smoke_test.py

# 3. 真实问题回归与阈值扫描
python scripts\qa_regression.py
python scripts\qa_regression.py --sweep

# 4. 更新知识库后重建索引
python scripts\build_index.py

# 5. 界面自动化验收（生成三张截图）
node scripts\ui_check.js http://127.0.0.1:8000
```

附录 B 上线前检查清单

- 按知识库中的核对清单逐项确认学校信息，重点是金额、电话、地址与政策条款
- 运行回归脚本，确认召回与跑题兜底均符合预期
- 把招生办提供的真实问题补进常见问答表并重建索引
- 确认管理令牌为随机长字符串
- 确认密钥文件未进入版本库，也未进入容器镜像
- 确认跨域来源已收紧为学校官网域名
- 确认部署进程数为 1，或已将会话与限流迁移至 Redis
- 对外话术经学校宣传部门审阅
- 全站启用 HTTPS